第4回 ― 自分のアプリからBlobに保存し、表とグラフで見る

アプリからCSVをBlobに保存し、保存したデータを表とグラフで表示します。 ページを開き直してもデータが残ることや、別のCSVを保存したときの違いを確認します。

第4回の座学後に、Microsoftから付与されたAzureサブスクリプションで取り組んでください。 RG、Storage、アプリ用のID、ネットワーク、ACR、Container Appを作成し、終了時には削除します。講師による環境の事前準備はありません。 アプリのソースと設定ファイルは教材に同梱しています。コードを書く必要はなく、配布ファイルを使って環境を構築します。

進め方: 座学後の自習。第1〜3回のリソースは不要
目安: **120〜150分＋削除待ち30分程度。**初回のビルド・ネットワーク作成で長くなることがあります
操作: Azure Portal、ブラウザーのCloud Shell（Bash）、作ったアプリ。PCにDockerを入れる必要はありません
更新日: 2026-10-13 ／ 共通の始め方と名前 ／ 実施記録

> 練習には同梱の架空CSVだけを使います。仕事のデータ、個人情報、秘密情報はアップロードしないでください。 Blobの公開アクセスとアカウントキーは使わず、アプリのマネージドIDで読み書きします。 IP制限は、ログイン認証の代わりではありません。本番のアプリでは利用者の認証とデータの分離も必要です。

画像には以前の動作確認時のものを使っています。リソース名や案内文が異なる場合は、本文の設定と確認値に従ってください。

目次

- 始める前の準備と費用
- 今日作るものと名前
- Exercise 1: RGと作業場所を用意する
- Exercise 2: 保存先とアプリを作る
- Exercise 3: 保存して、開き直して比べる
- Exercise 4: 自分のリソースを片付ける
- 完了チェックと振り返り
- 困ったとき

始める前の準備と費用

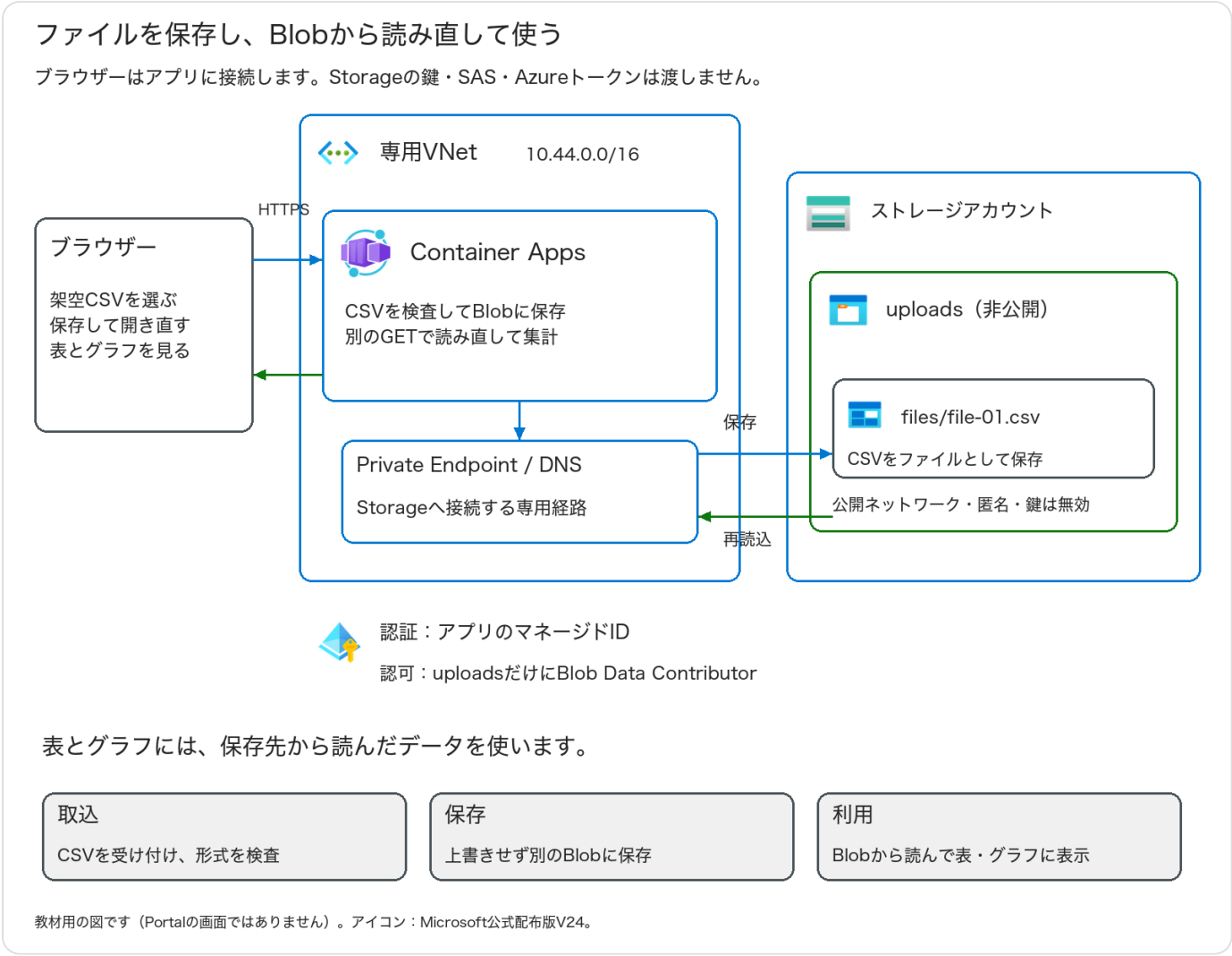
- [ ] Microsoftから付与されたサブスクリプションにサインインできる。
- [ ] RGと今回のリソースを自分で作成・削除できる。
- [ ] 新しいBlobコンテナーとACRで、アプリのIDへRBACロールを割り当てられる。
- [ ] 個人識別名を決め、費用の扱い、利用期限、問い合わせ先を控えた。
- [ ] Japan Eastで今回のサービスを使え、必要なプロバイダーが登録されている。
- [ ] PCのブラウザーからPortalとHTTPSのアプリを開ける。
- [ ] 同梱の架空CSVだけを使う。

Contributorだけでは、通常ロール割り当てはできません。 第4回ではデータへの権限も設定するので、その許可が必要です。 権限が足りない場合は、今回作るBlobコンテナーとACRへのロール割り当てが必要であることを付与元に伝えてください。 権限の問題を避けるために、ポリシーを解除したりStorageのキーを使ったりしないでください。

必要なプロバイダーは`Microsoft.App`、`Microsoft.Network`、`Microsoft.Storage`、`Microsoft.ManagedIdentity`、`Microsoft.ContainerRegistry`、`Microsoft.CloudShell`です。Portalの**サブスクリプション→リソースプロバイダー**で確認できます。未登録のものは、付与された許可の範囲で登録します。登録できなければ付与元に相談してください。

料金はCSVの容量だけでは決まりません。Container Apps、Storageの操作、ACRとビルド、Private Endpoint、Private DNS、 VNet統合で作られるロードバランサーやIPも費用の対象です。 公開小売価格の参考値は、ACR Basicが$0.1666/日、Private Endpointが$0.01/時、Hot LRS保存が$0.02/GB・月でした（2026-10-12、USD）。 実際の請求額は、契約、無料枠、保存期間、転送量、税などによって変わります。この参考値は実習費用の見積もりや上限額ではありません。 始める前に終了時刻を決めてください。途中でやめる場合も、作成済みのRGは削除します。

今日作るものと名前



ブラウザーはアプリに接続します。Blobへの保存と読み込みは、アプリが行います。 Container Appsの「コンテナー」はアプリを実行するもの、Blobの「コンテナー」はファイルをまとめるものです。 名前は同じですが、役割は異なります。

`ksaito73` は個人識別名の例です。共通ルールで決めた自分の名前に置き換えます。

作るもの	名前の例・役割
RG	`rg-hol04-ksaito73` 。自分でPortalから作る
Container App	`ca-hol04-ksaito73` 。CSVを受け取るアプリ
Container Apps環境	`cae-hol04-ksaito73` 。アプリの実行場所
マネージドID	`id-hol04-ksaito73` 。アプリがAzureへ接続するID
VNet	`vnet-hol04-ksaito73` 。アプリとBlobの専用経路
Private Endpoint	`pe-hol04-ksaito73-blob` 。Blobへつなぐ入口
Storage	`st04ksaito73...` 。個人識別名＋重複を避ける文字列を自動生成
ACR	`acr04ksaito73...` 。自分でビルドしたイメージの保存先
Blobコンテナー	`uploads` 。このアプリのCSVだけを保存
Private DNS	`privatelink.blob.core.windows.net` 。指定名なので変えない

アプリは64 KiB以内・1〜24行のCSVを10件まで保存します。 アップロード後、アプリはBlobからCSVを読み込み、その内容を表とグラフに表示します。ブラウザー内に残ったデータを表示するだけではありません。CSVはファイルとして保存します。SQLやRedisは使いません。

Exercise 1: RGと作業場所を用意する

Task 1 — 付与されたサブスクリプションと名前を確認する（5分）

1. Azure Portalへ、案内されたアカウントでサインインします。
2. **サブスクリプション**を検索し、Microsoftから付与されたものの名前を確認します。
3. この回ではコマンドにも指定するので、概要の**サブスクリプションID**を手元に控えます。教材や提出物には載せません。
4. 個人識別名と `rg-hol04-自分の識別名` を控えます。前回の続きから再開する場合は、保存した作業記録を復元します。

Task 2 — 自分の空のRGを作る（5分）

1. **リソースグループ→作成**を開きます。
2. 付与サブスクリプション、 `rg-hol04-自分の識別名` 、許可された**Japan East**を選びます。

3. **タグ**に `Purpose=AzureHandsOn` 、 `Session=04` 、 `Participant=自分の識別名` を入れます。
4. **確認および作成→作成**をクリックし、自分のRGの概要を開きます。
5. 中が空で、名前とタグが自分のものだと確認します。

後の手順で使うスクリプトは、このRGの名前とタグを確認します。初回の実行では、空でないRGや他の参加者のRGを変更しない仕組みになっています。

Task 3 — 配布ファイルをCloud Shellに入れる（5〜10分）

1. 第4回の作成ファイル一式をPCへダウンロードします。 **HTML版のこのリンクを使ってください。** PDF自体にはZIPは入っていません。
2. Portal上部の **Cloud Shell**（ `>_` のアイコン）を開き、**Bash**を選びます。
3. 初めて使う場合は、**No storage account required／ストレージアカウントは不要**を選び、付与サブスクリプションで開始します。
4. **Manage files／ファイルの管理→Upload／アップロード**から、ZIPをアップロードします。
5. Cloud Shellの入力欄に、次をコピーしてEnterを押します。

```
unzip handson04-self-study.zip
cd handson04-self-study
```

Cloud Shellはブラウザー内の作業場所です。PCのターミナルや、前の回のVMには入力しません。 ZIPにはアプリのソース、Bicepの設定、 `setup_lab.py` が入っています。イメージのビルドとAzure環境の作成は、これから行います。

このCloud Shellは一時セッションです。閉じたり再起動したりすると、ファイルが消えます。 処理を待っている間でも、20分程度操作がないと終了する場合があります。後の手順に従い、作業記録を各段階でダウンロードしてください。

Task 4 — 自分の作成先とPCのIPを記録する（5分）

1. **PCのブラウザーの別タブ**でIPv4を確認するページを開き、数字を手元に控えます。 Cloud Shellで調べるIPは別になることがあるので、ここはPCのブラウザーで確認します。
2. 次の3か所を自分の値に置き換え、Cloud Shellへ入力します。

```
python3 setup_lab.py init \
  --subscription "自分のサブスクリプションID" \
  --id "ksaito73" \
  --client-ip "PCのブラウザーで確認したIPv4"
```

`ksaito73`も自分の識別名へ置き換えます。日本語の説明部分を残したまま実行しないでください。 出力の「今回の対象」に、控えたRG名と識別名が表示されていることを確認します。 アプリへのアクセスは、ここで指定したIPだけに制限します。IPが変わった場合は作業を止めて設定を確認し、すべてのIPからのアクセスは許可しないでください。

作業記録の保存: Cloud Shellで`pwd`を入力して、今のフォルダーの場所を控えます。 Manage files→Downloadから、その場所の`.lab04-state.json`をPCへ保存してください。 例は`/home/`**自分のユーザー**`/handson04-self-study/.lab04-state.json`です。記録には名前・ID・IP・実行番号が入りますが、キーやトークンは入りません。手元で保管し、提出しません。

Exercise 2: 保存先とアプリを作る

Task 5 — Storage、ID、ネットワーク、実行環境を作る（15〜30分）

```
python3 setup_lab.py prepare
```

スクリプトはARMの事前チェックとwhat-if（変更内容の事前確認）を実行し、今回のRG内の新規作成だけであることを確認してから展開します。 作成するのは、Storage、uploadsコンテナー、マネージドID、ACR、VNet、Private Endpoint、DNS、Container Apps環境です。

マネージドIDには、**uploadsコンテナーだけ**のStorage Blob Data Contributorと、**自分のACRだけ**のAcrPullを割り当てます。 Storageは非公開、Shared Keyと匿名アクセスも無効です。環境はConsumptionだけ、ログ保存先は追加しません。

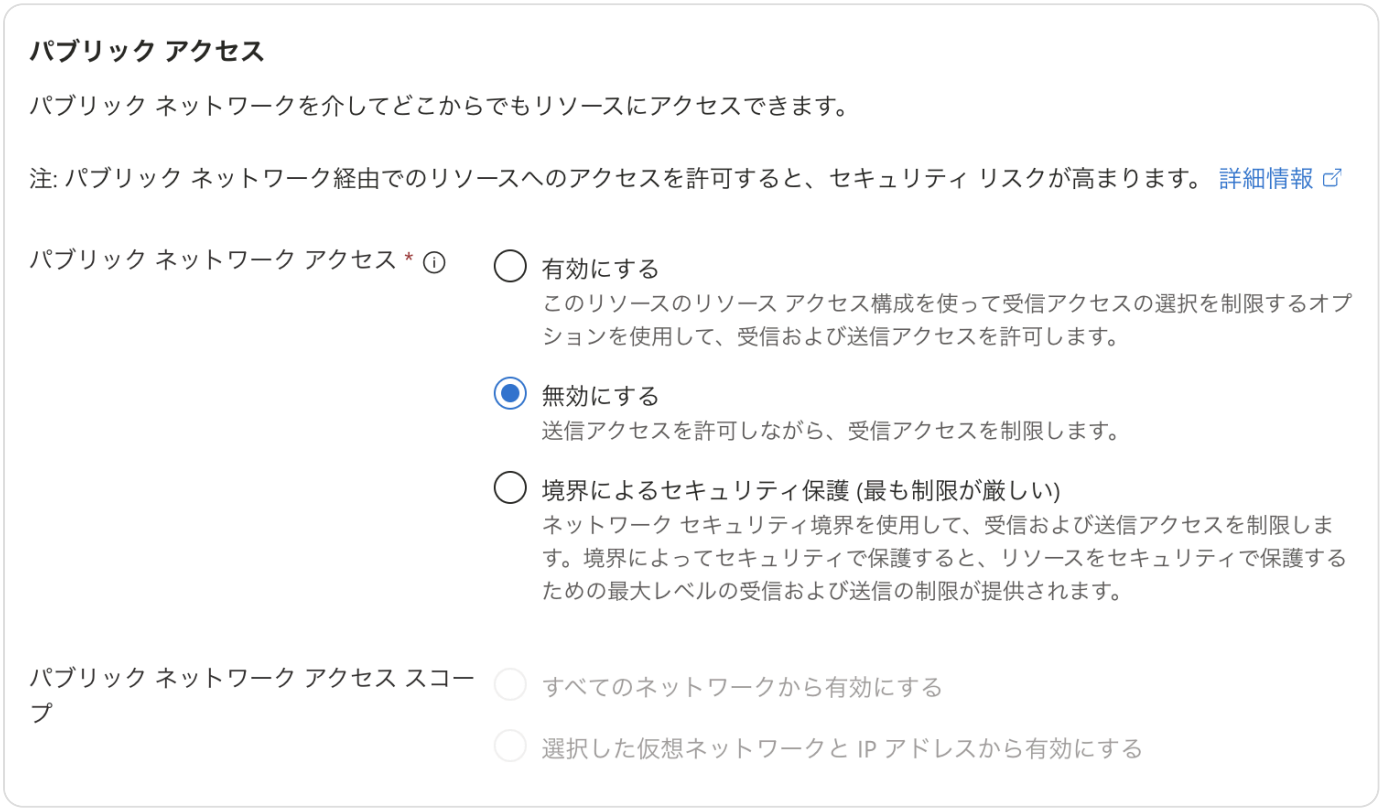
名前の一覧と「基盤を作成しました」が出るまで待ちます。 待ち時間の上限でコマンドが終了した場合は、同じ`prepare`を再実行します。作業記録を基に、実行中のデプロイの状況を確認できます。別のRGでやり直す必要はありません。 完了後に`.lab04-state.json`をもう一度保存します。

Task 6 — 作ったリソースをPortalで見る（10分）

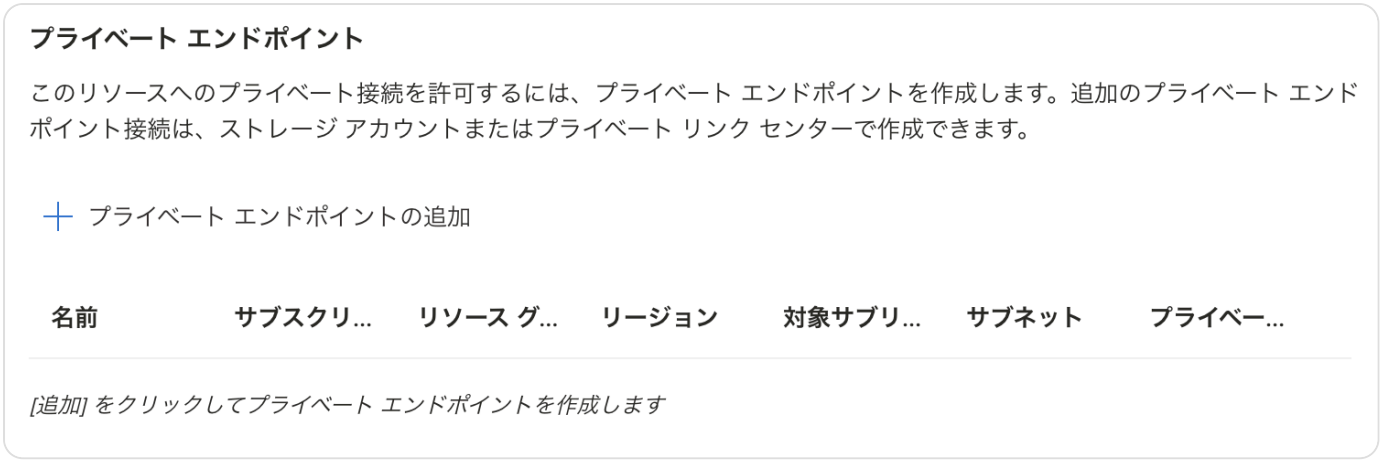
1. Cloud Shellを最小化し、**自分のRG→概要**を更新します。
2. 名前に自分の識別名を含むアプリ用ID、VNet、環境、Storage、ACR、Private Endpointを探します。
3. Storageの**構成／Configuration**で、匿名アクセスとShared Keyを使わない設定を確認します。
4. **ネットワーク**で、公開ネットワークが無効、Blob用Private Endpointが承認済みであることを見ます。
5. アプリ用IDに付いた権限の範囲が、今回のuploadsコンテナーとACRであることを見ます。表示できなければ、通知と付与権限を確認します。



画像は**作成前の設定画面**です。作成済みのStorageでは、対象リソースの**構成**を開いて確認してください。



公開ネットワークを無効にしたStorageへアプリから接続するには、Private Endpoint（PE）、DNS、VNet統合も必要です。



画像はPrivate Endpointの追加前なので、一覧は空です。手元のStorageでは、`pe-hol04-...-blob`が作成されていることを確認してください。 PCからPrivate Endpointへの接続経路は用意していないため、PortalではBlobの内容を直接開けない場合があります。ファイルの操作には、後で展開するアプリを使います。

Task 7 — 自分のACRでアプリをビルドする（5〜15分）

Cloud Shellに戻り、次を実行します。

```
python3 setup_lab.py build
```

配布されたソースを自分のACRへ送り、Azure上でLinuxのイメージをビルドします。PCでDockerを動かす必要はありません。 「ビルド...が完了しました」が出たら、作業記録をもう一度保存します。

ビルドの完了前にコマンドが終了した場合は、同じ`build`を再実行します。記録された実行番号（runId）の状況を確認するため、ビルドが重複して起動することはありません。 失敗した場合は、ACRの**Tasks→Runs**で対象の実行を確認してください。ソースや作業記録を失った場合は、同名のリソースへ上書きせず、困ったときを参照してください。 ビルドは受け付けられたものの実行番号を記録できなかった場合は、ACRで同じイメージタグのビルドが成功していることを確認し、`python3 setup_lab.py build --resume-run 実行番号`で作業記録に登録します。指定した実行のタグが今回のタグと一致しなければ、スクリプトは処理を中止します。

Task 8 — Container Appを展開する（5〜15分）

```
python3 setup_lab.py deploy
```

ビルドしたイメージをdigest（イメージを一意に識別する値）で指定し、Container Appを作成します。アプリはマネージドIDでACRからイメージを取得します。 設定は0.25 vCPU／0.5 GiB、最小0・最大1レプリカです。ブラウザーからはHTTPSで接続し、アプリへの転送先はポート8080です。アクセスはTask 4のIPだけに制限します。 Storageのキーや接続文字列は入力しません。

表示された**自分のアプリURL**を手元へ控え、ブラウザーの別タブで開きます。 マネージドIDの権限が反映されるまで、時間がかかる場合があります。アプリを開けたら、次の手順でCSVの保存と読み込みを確認します。

Exercise 3: 保存して、開き直して比べる

Task 9 — 自分のアプリとサンプルを開く（5分）

1. URLで「CSVをBlobに保存」する画面が開くことを見ます。
2. 「ローカルテスト用・Azureには保存しません」と出ていないことを確認します。出ていたら別の画面なので、作成先を見直します。
3. 保存済み0件、または今回自分で保存したファイルだけがあることを見ます。
4. アプリの**基本CSV**と**比較CSV**のリンクからファイルを保存します。



自分のURLはTask 8で作ったものです。講師からアプリURLを受け取る必要はありません。 資料にも基本CSV、比較CSVを同梱しています。 Excelで保存し直さず、そのまま使ってください。

確認値	基本CSV	比較CSV
データ行数（ヘッダーを除く）	6	6
合計	1020	1080
最大値	240	300
違い	—	6月の240を300へ変更

Task 10 — 基本CSVを保存し、表とグラフを見る（5分）

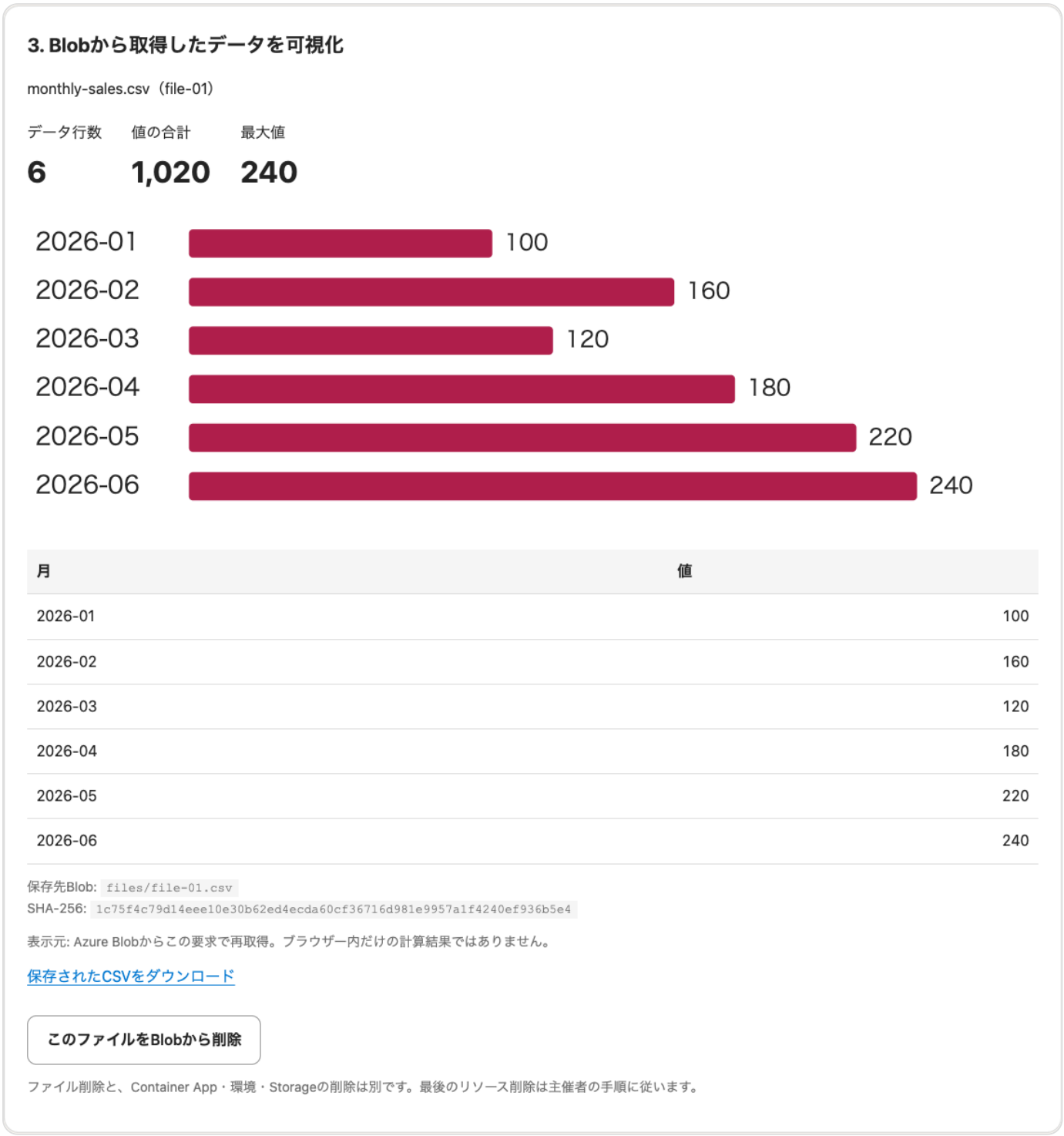
1. ファイル入力で `monthly-sales.csv` を選び、**Blobへアップロード**を一回クリックします。

2. 保存と表示が終わるまで待ち、ファイルIDを控えます。

3. **6行・合計1020・最大240**を確認します。

4. 表の100、160、120、180、220、240と、6本の棒を見比べます。

5. 表示元がAzure Blobであること、Blob名とSHA-256が出ることを確認します。



画像の`file-01`は例です。手元に表示されたファイルIDを選んでください。 小さい画面ではグラフの部分を横にスクロールでき、下の表でも値を読めます。 保存成功後に表示が失敗した場合は、再アップロードを繰り返さず一覧を再読み込みします。

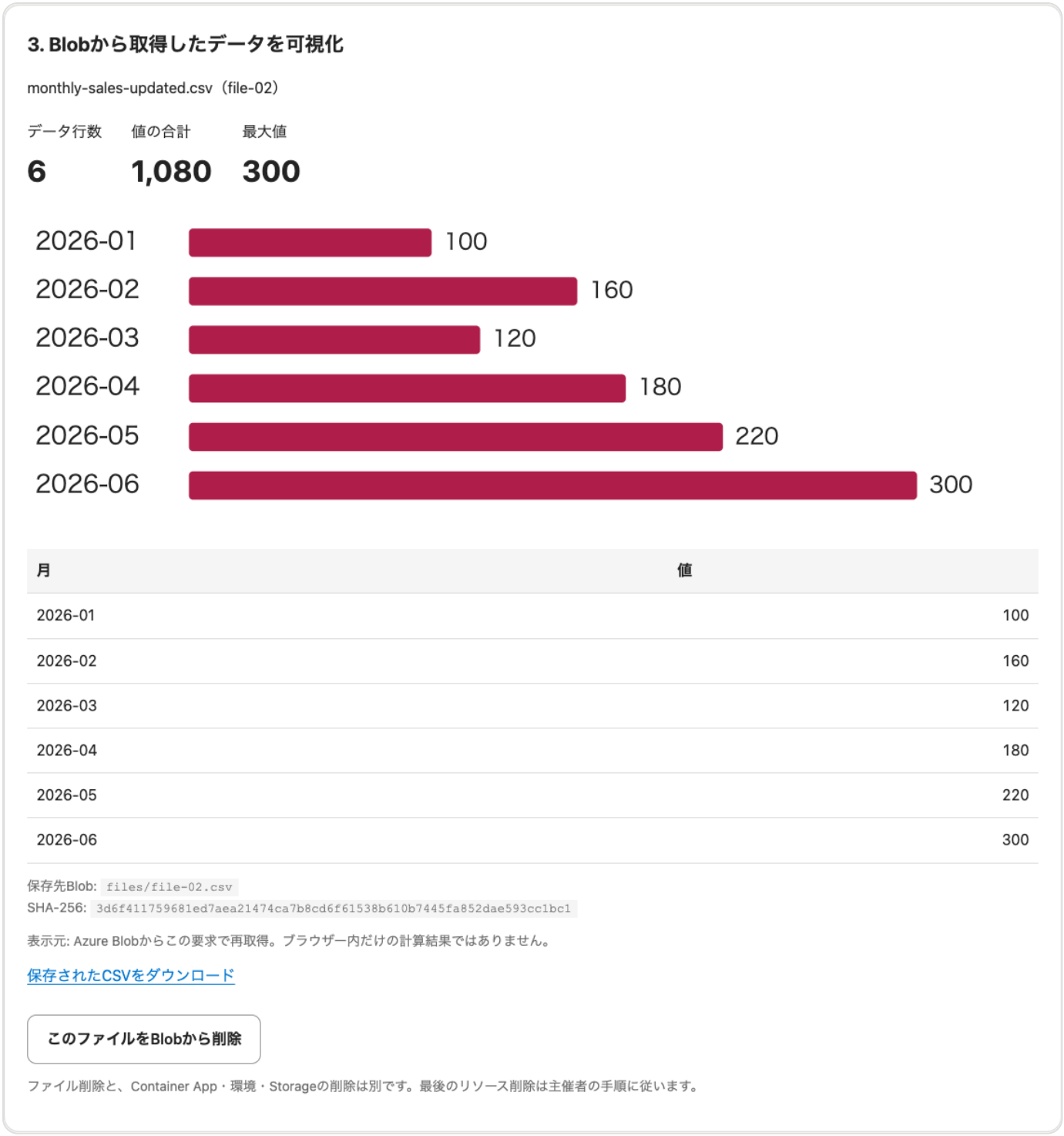
Task 11 — ダウンロードと開き直しを試す（5分）

1. **保存されたCSVをダウンロード**をクリックし、元の6行と一致することを見ます。
2. **Blobから再読み込み**をクリックします。
3. ページ自体も開き直し、同じファイルIDを選びます。可能なら別のプライベートウィンドウも使います。
4. 6行・1020・240が同じであることを確かめます。

Blobに保存したファイルを、ページを開き直した後も読み込めることを確認する手順です。 以前の検証時の画面も参考にしてください。

Task 12 — 比較CSVを保存し、元のファイルと比べる（5分）

1. `monthly-sales-updated.csv` を保存し、別のファイルIDになったことを見ます。
2. 6月が300、合計1080、最大300であることを確認します。
3. 最初のIDへ戻し、合計1020、最大240のままであることを確かめます。



同名でも別の空きIDに保存するので、元のCSVは上書きされません。選ぶときはファイル名とIDの両方を見ます。

Task 13 — 違う列名のCSVが保存されないことを試す（5分）

1. 列名が違うCSVを使います。先頭行が `date,total` のサンプルです。
2. 保存済み件数を控えてから、そのCSVをアップロードします。
3. 「先頭行はmonth,valueの2列にしてください」というエラーを見ます。
4. 一覧を読み直し、件数が増えていないことを確認します。

先頭行はmonth,valueの2列にしてください。

受け付けるのはUTF-8（BOM可）、64 KiB以内・1〜24行、2000〜2099年の重複しない月、0〜1000000の整数です。 負数、小数、式、追加列は受け付けません。試すのはこの架空の小さいサンプルだけにします。

Exercise 4: 自分のリソースを片付ける

Task 14 — 保存したファイルをアプリから削除する（5分）

1. 自分が保存したファイル名とIDを選び、**このファイルをBlobから削除**をクリックします。
2. 確認画面の対象を読んで削除します。
3. 比較CSVも同様に削除し、開き直して保存済み0件を確かめます。



削除時にはETagを使い、表示してからファイルが変更されていないことを確認します。変更を知らせるメッセージが出た場合は、一覧を再読み込みしてください。 ファイルを削除しても、アプリ・ACR・Storageは残ります。次の手順で削除します。

Task 15 — 自分のRGを削除する（5〜30分以上）

1. Portalで**自分のRG**を開き、付与サブスクリプション、RG名、`Participant`タグを控えと照合します。
2. 中のStorage、ACR、ID、VNet、PE、DNS、環境、アプリが、今回自分で作ったものだけだと確認します。
3. **リソースグループの削除**をクリックし、対象を読み、自分のRG名を入力して削除します。
4. 通知で処理状況を確認します。途中で作成に失敗した場合も、この片付けは必要です。

削除は取り消せません。 共有・業務用・他人のRGは選びません。 自分の環境が作った`ME_...`の管理用RGは、環境削除に伴って片付きます。直接編集・削除する操作はしません。 削除には20分以上かかることがあります。開始通知だけで完了にせず、次の一覧まで確認してください。

Task 16 — RG・アプリ・環境が消えたことを確認する（5分）

1. **リソースグループ**一覧を更新し、自分のRG名で検索します。
2. **RGが表示されない**ことを確認します。
3. **Container Apps**と**Container Apps環境**、StorageとACRの一覧でも、自分の名前のリソースがないことを確認します。
4. Cloud Shellの記録が残っていれば、次も実行します。

```
python3 setup_lab.py check-cleanup
```

このコマンドは、作業記録にある実習用RGと管理用RGが削除されたことを確認します。 まだ残っている場合は通知を確認し、時間を置いて一覧を更新してください。権限やロックのエラーは付与元に相談し、解消後に削除を再開します。 削除が終わるまでは「未完了」と記録してください。請求画面の0円表示ではなく、対象リソースが一覧からなくなったことをもって完了とします。

完了チェックと振り返り

- [ ] Microsoftから付与されたサブスクリプションを使った。
- [ ] 個人識別名のRGを自分で作り、Participantタグを付けた。
- [ ] Storage・ID・権限・ネットワーク・ACR・環境を自分で作った。
- [ ] 配布ソースを自分のACRでビルドし、自分のContainer Appへ展開した。
- [ ] 基本CSVはBlobから読み直した結果が6行・1020・240だった。
- [ ] ダウンロードと開き直し後も、同じファイルを確認できた。
- [ ] 比較CSVは1080・300、元のCSVは1020・240のままだった。
- [ ] 列名の違うCSVは拒否され、保存件数が増えなかった。
- [ ] 自分のファイルを削除した。
- [ ] 自分のRGを削除し、アプリ・環境・Storage・ACRが残っていないことを確認した。

実施結果、未完了のTask、CSVの確認値、削除状況、操作で迷った箇所を提出してください。実際のURL・IP・IDや作業記録ファイルは含めないでください。 アプリの画面表示までなのか、CSVの保存・再読み込みまで確認できたのかが分かるように記録します。

問い1: ブラウザー、アプリ、Blobは、それぞれどのような役割でしたか。
問い2: アプリのIDに与えるデータの操作権限を、uploadsコンテナーだけに限定したのはなぜですか。
問い3: ファイルを削除した後も、どのリソースが残っていましたか。

困ったとき

止まったところ	次に確認すること
RGを作成できない／ロールを割り当てられない	サブスクリプションと権限を確認してください。Contributorだけでは、通常ロールを割り当てられません。
プロバイダー未登録・Policy・Quotaのエラー	プロバイダーの登録状況、許可されたリージョン、クォータを確認します。必要な権限がない場合は付与元に相談してください。
Cloud ShellでZIPが見つからない	Manage filesからアップロード済みか確認してください。展開後は、`cd handson04-self-study`で作業フォルダーに移動します。
一時セッションを閉じて作業記録が消えた	ZIPを再度展開し、PCに保存した`.lab04-state.json`を同じフォルダーへアップロードしてください。既存RGに対してinitをやり直さないでください。
作業記録を保存する前に失った	Portalで作成済みのリソースを確認してください。重複して作成せず、今回のRGを削除してから最初からやり直します。
prepareが失敗した	今回のRGのデプロイ詳細でエラーを確認してください。別のRGや公開Storageで代用しないでください。
buildの受付状況が分からない／失敗した	ACRのTasks→Runsで実行結果を確認してください。成功していて実行番号だけが未記録の場合は、`build --resume-run` **実行番号** で記録に登録します。失敗した場合は原因を記録し、続行できなければRGを削除してください。
アプリURLで403が返る	PCの現在のIPv4と、アプリのイングレスのIP制限を確認してください。IPが変わっていた場合は、今回のアプリのclassroom規則を現在のIPv4（/32）に置き換えます。すべてのIPからのアクセスは許可しないでください。
Storageへのアクセスが拒否される	アプリのマネージドID、コンテナー単位のRBAC、権限の反映状況、Private EndpointとDNSを確認してください。
PortalからBlobを開けない	PCにはPrivate Endpointへの接続経路がないため、そのままで問題ありません。アプリから確認し、キーやSASは発行しないでください。
保存は成功したが表示に失敗した	一覧を再読み込みしてください。アップロードを繰り返す必要はありません。
削除が進まない	通知と、今回のRG・環境の状態を確認してください。管理用RGは直接削除せず、付与元に相談します。

Microsoft公式の補足

- 一時Cloud Shellを始める
- Cloud ShellのUpload／Download
- ACRでのイメージビルド
- Blobデータの認証と認可
- Container AppsのマネージドID
- StorageのPrivate Endpoint
- Container AppsのVNet統合と管理用リソース